Document d'Étude et Conception (DEC)

Marcel's Company

Project M2D

Author: Marcel's Company
Version: 09/03/2026
Description: Modeling of a Simple Maintenance Model

I. **General Vision** **3**
A. Objectives 4
B. Key Performance Indicators (KPIs) 4
II. **Simulator (SimMCM2D)** **7**
A. Dataset Structure 7
B. Simulated Cases 8
C. Package Uses 9
D. User Interface 9
III. **Experiments** **12**
A. Exploratory Analysis 12
B. Parameter Impacts 16
IV. **Mathematical Description of Models** **32**
V. **Parametric Estimation** **32**
VI. **Policy Optimization** **32**
A. Cost Function C(θ) 32
B. Monte-Carlo estimation of the cost 33
C. Genetic Algorithm 34
D. Two-Stage Evaluation 37
VII. **Appendix** **39**
A. Inspection Quality Law 39
B. Fleet Evolution Law 40

I. General Vision

The project consists of a simple fleet maintenance model. It simulates the real problem faced by airline agencies and similar businesses in analyzing and optimizing their maintenance strategies. In this project, this is done in a simplified controlled scenario, where a single component is considered, and data is simulated. The codes, analysis, and results will be made available in the private GitHub repository: https://github.com/alc4ml/ProjetM2D2526/

The project comprises different objectives, which are described as follows:

A. Objectives

We describe the general objectives to be achieved by the project:

1. **Create a Simulator:** No real data is available, so we propose and implement the necessary tools to simulate the scenarios and generate datasets that resemble the real scenarios of interest. This way, different scenarios and strategies can be proposed and studied without relying on real experiments.
2. **Inverse Problem:** Given a fixed dataset generated by the previous simulator, we are interested in recovering/estimating the underlying parameters used to generate the sample. This unique dataset represents the real dataset received by the company for the study, and the parameters estimated will be used to extrapolate this context.

3. **Optimize Strategies:** From the previous study, we extrapolate the observed dataset by maintaining fixed parameters (e.g., component wear rate, fleet growth rate, etc.) untouched and adjusting strategic maintenance parameters (e.g., frequency and precision) in order to propose improvements.

Besides those main objectives, several intermediate objectives are also highlighted, such as steps of Exploratory Analysis, Parameter Impact Analysis, Proposition of Key Performance Indicators (KPIs), and key Cost Functions/Metrics to be optimized.

B. Key Performance Indicators (KPIs)

A Key Performance Indicator (KPI) is a quantitative measure used to evaluate the effectiveness and success of an organization, a system, or a strategy in relation to predefined objectives. In the context of industrial maintenance, KPIs help to manage equipment reliability, optimize operational costs, and measure the impact of preventive strategies. KPI stands for key performance indicator, a quantifiable measure of performance over time for a specific objective. KPIs provide measurable targets for teams, allow progress to be tracked over time, and generate insights that support better decision-making across the organization.

In this analysis, three main categories of KPIs were defined: the number of failures and replacements, the quality of the detector and the costs.

1/ Number of failures and replacements

Several indicators were calculated to evaluate system reliability across the entire fleet. At the fleet level, the number of failures and replacements was calculated over three different time periods: monthly (m), quarterly (t) and yearly (y). The objective is to determine how frequently systems fail and how often they require repair or replacement.

First, we aim to measure the average number of failures across the entire fleet. To do this, the dataframe is grouped by period (m, t, or y). The total number of failures is then calculated for each period, and the average of these values is subsequently determined. For example, to obtain the average number of failures per month, the average of the number of failures observed across all months is calculated. The corresponding indicators are:

- **avg_fail_m** : Average number of failures per month
- **avg_fail_t** : Average number of failures per trimester
- **avg_fail_y** : Average number of failures per year

The same principle is applied to replacements, using the variables: **avg_repl_m, avg_repl_t** et **avg_repl_y.**

The standard deviation is also calculated for these indicators. It makes it possible to measure the stability of the fleet's maintenance activity. For example, in the case of failures:

- a low standard deviation indicates that the number of failures is relatively constant from one period to another
- a high standard deviation indicates a high variability in the number of failures.

These indicators are stored in **std_fail_** et **std_repl_** .

In a second step, the objective is to determine how many times a system fails on average, either over the entire lifetime of the fleet or per period.

- **avg_fail_syst_total**: Average over the total lifetime of the fleet: first, the total number of failures is calculated for each individual system, and then the average of these totals is computed.

- **avg_fail_syst_**: Average per system and per period. After this grouping, we obtain for example a data structure such as:
    - January – System 1: 2 failures
    - January – System 2: 0 failures
    - February – System 1: 1 failure
    - etc.

The average of these values is then calculated in order to obtain the average number of failures per system and per period.

- **avg_repl_sys**: the same calculation principles are applied to replacements.

Finally, we compute the **PER**, Preventive Effectiveness Ratio, which is used to evaluate the balance of the maintenance strategy:

- *PER > 1* **:** More preventive replacements are performed than actual failures occur. If the ratio is very high, components may be replaced too early (over-maintenance).
- *PER < 1 :* There are more failures than replacements. This indicates that preventive maintenance is not frequent enough or not effective enough to prevent failures (under-maintenance).
- *PER ≈ 1 :* There is a numerical balance between planned interventions and observed failures.

2/ Quality of the detector

The quality of the detection system is evaluated by analyzing its ability to correctly identify failures. A detection is counted when an inspection leads to a valid replacement, meaning that the alert actually corresponds to a real failure.

Two types of events are considered:

- Detection: corresponds to the number of times the system correctly identified an event (true positives). The flag FF (Failure Found) set to True indicates that the alert was justified.
- False alerts: corresponds to the number of times the system triggered an alert when no real failure was present (false positives).

From these two metrics, two standard indicators are calculated.

- **Precision** answers the question: *"When the system triggers an alert, what is the probability that there is actually a failure?"*

It is calculated as follows: $Precision = \frac{Detection}{Detection + False\ alerts}$

A precision of 1 (100%) means that there are no false alerts. Every triggered intervention is useful.

- **Recall** answers the question: *"Out of all the failures that occurred, what percentage was successfully detected by the system?"*

It is calculated as follows: $Recall = \frac{Detection}{Total\ nb\ of\ failures}$

For example, if Recall = 0.5, it means that only one failure out of two is detected by the system.

3/ Costs

The last category of KPIs concerns the analysis of the fleet's maintenance costs.

- **cost_per_syst:** average cost of a system across the entire fleet. To calculate it, the dataframe is grouped by system_id (that is, by system). The total cost is then calculated for each system.

For example, for a fleet composed of two systems: System 1 costs €3000 and System 2 costs €1000. The average cost per system is therefore (3000 + 1000) / 2 = €2000. This grouping also makes it possible to calculate the standard deviation of the cost per system, in order to evaluate the variability of costs between the different systems.
Next, we calculate the average fleet cost for different periods.

- **cost_fleet_**: the dataframe is grouped by period (m, t, or y). For each period, the total fleet cost is calculated. This produces a series of costs per period. The average of these costs is then computed: $\frac{M_1+M_2+...+M_n}{n}$

where M_i represents the total fleet cost for a given period.

Finally, we calculate:

- **avg_cost_syst_ :** the average cost of a system per month, quarter, and year. The dataframe is grouped by period and by system_id. For each period, the total cost of each system is calculated, and an average is computed in order to obtain the average cost of a system during that period. The average of these values across all periods provides the average cost per system for a month, a quarter, or a year.

II. Simulator (SimMCM2D)

The *Simulator Marcel's Company M2 des Données (SimMCM2D)* is implemented in Python, taking advantage of its mature development and deployment environments available across many platforms. A complete set of installation and use instructions is available in the

repository. In particular, the library can be installed using the established Pip Install Packages (PIP) manager.

A. Dataset Structure

The simulator produces files that record the operational and maintenance history of each system. The following table shows a representative extract of the data currently generated:

	system_id	component_id	event_date	event_time	event_type	event_report	system_age	component_age	usage_since_last_event_h	FF	cost_event	cost_cumulated
0	1	1	2010-01-08	20:35:21	inspection	NaN	188.59	188.59	188.59	NaN	100	100
1	1	1	2010-01-15	15:12:59	inspection	NaN	351.22	351.22	162.63	NaN	100	200
2	1	1	2010-01-21	21:12:00	inspection	NaN	501.20	501.20	149.98	NaN	100	300
3	1	1	2010-01-23	15:20:16	failure	NaN	543.34	543.34	42.14	NaN	1200	1500
4	1	1	2010-01-23	15:20:16	replacement	unused	543.34	543.34	0.00	NaN	1100	2600
5	1	2	2010-01-29	09:54:39	inspection	NaN	681.91	138.57	138.57	NaN	100	2700
6	1	2	2010-02-04	18:07:06	inspection	NaN	834.12	290.78	152.21	NaN	100	2800
7	1	2	2010-02-10	20:52:03	inspection	NaN	980.87	437.53	146.75	NaN	100	2900
8	1	2	2010-02-18	16:05:40	inspection	NaN	1168.09	624.76	187.23	NaN	100	3000
9	1	2	2010-02-22	04:51:08	failure	NaN	1252.85	709.51	84.76	NaN	1200	4200

In the example, each row represents an independent event recorded by a manager. The fields on the resulting dataset are described as follows:

- **system_id:** unique identifier of the system, issued by the factory;
- **component_id:** unique identifier of the component, issued by the factory;
- **event_date:** date (year, month, and day) when the event happened;
- **event_time:** the time (hour, minutes, and seconds) on the day the event happened;
- **event_type:** one of the events that can happen: *inspection*, *replacement*, or *failure*
- **event_report:** further information recorded during an event:
    - **inspection reports:** an inspection may result in replacement for the following two reasons: the mandatory replacement age was reached (expired); or the detector identified aging-related issues (wornout);
    - **replacement reports:** a replacement may substitute an old component with a new component from the factory (unused) or reuse a previously replaced component detected to be in good condition (reused);
- **system_age:** the age of the system, measured in hours of use;
- **component_age:** the age of the component, measured in hours of use;
- **usage_since_last_event_h:** the amount of time the system has been used since the last event related to it, measured in hours of use;
- **FF (failure found):** the report of deeper inspection carried out to check if a component replaced can still be reused;
- **cost_event:** the cost incurred by the event;
- **cost_cumulated:** the total cumulated cost up to that moment

B. Simulated Cases

The simulator produces a timeline of events for each system, taking into account component aging, scheduled inspections, and real failures. The main scenarios are the following.

- **Inspection without Replacement:** An inspection is carried out at the planned date. If the estimated failure risk remains below the defined threshold, no replacement is

performed. The component continues to operate normally, and only the inspection cost is recorded.

- **Inspection Leading to Preventive Replacement:** During an inspection, the failure risk exceeds the allowed threshold. A preventive replacement of the component is then triggered. After this replacement, a deep inspection is carried out to evaluate whether the decision was justified or not. This analysis makes it possible to identify correct replacements as well as decision errors.
- **Component Reuse and Maintenance:** An inspection may imply a later replacement, and after replacement, a deeper inspection is carried out. If the removed component was identified as still in good condition, it is saved for later reuse. In the same process, the component is restored to a like-new condition. The cost of reusing a component is inferior to the cost of acquiring a new one.
- **Real Failure (Pane):** A failure occurs before the next scheduled replacement. The system suffers a breakdown, and immediate replacement of the component is required. The failure cost is recorded, and a new component is installed to restore the system operation.
- **Inspection Scheduling:** After each inspection, a new inspection date is automatically scheduled. This date depends on the chosen maintenance strategy (for example, frequent or spaced inspections). The time interval between two inspections has a direct impact on the failure risk: the more spaced the inspections are, the higher the probability that a failure occurs before the next one.
- **Automatic Replacement at Age Limit:** A component is automatically replaced when it reaches a predefined maximum age, independently of inspections.
- **Delay Before Replacement:** In some situations, a component cannot be replaced immediately after a failure or a maintenance decision. A delay is then taken into account before the intervention, potentially leading to additional costs.
- **Fleet Evolution Over Time:** The number of systems also evolves over time, following a random law that includes the creation of new systems, but also the removal of systems.

Further description of the laws and parameters for these scenarios is provided in the Appendix.

C. Package Uses

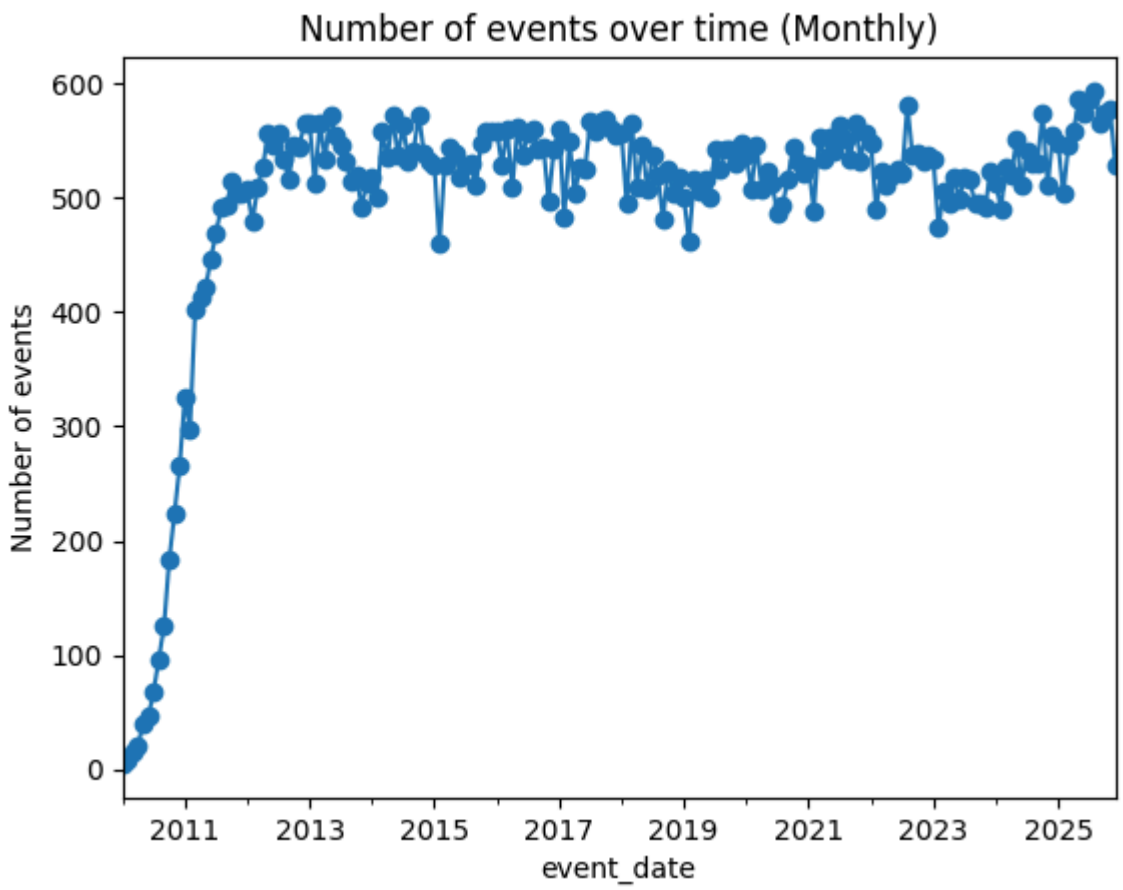
The main features implemented by the library are the following:

Further experiments show promising performance, as we were able to generate random samples of varied lengths, with various numbers of systems, in relatively short times, as shown in the table:

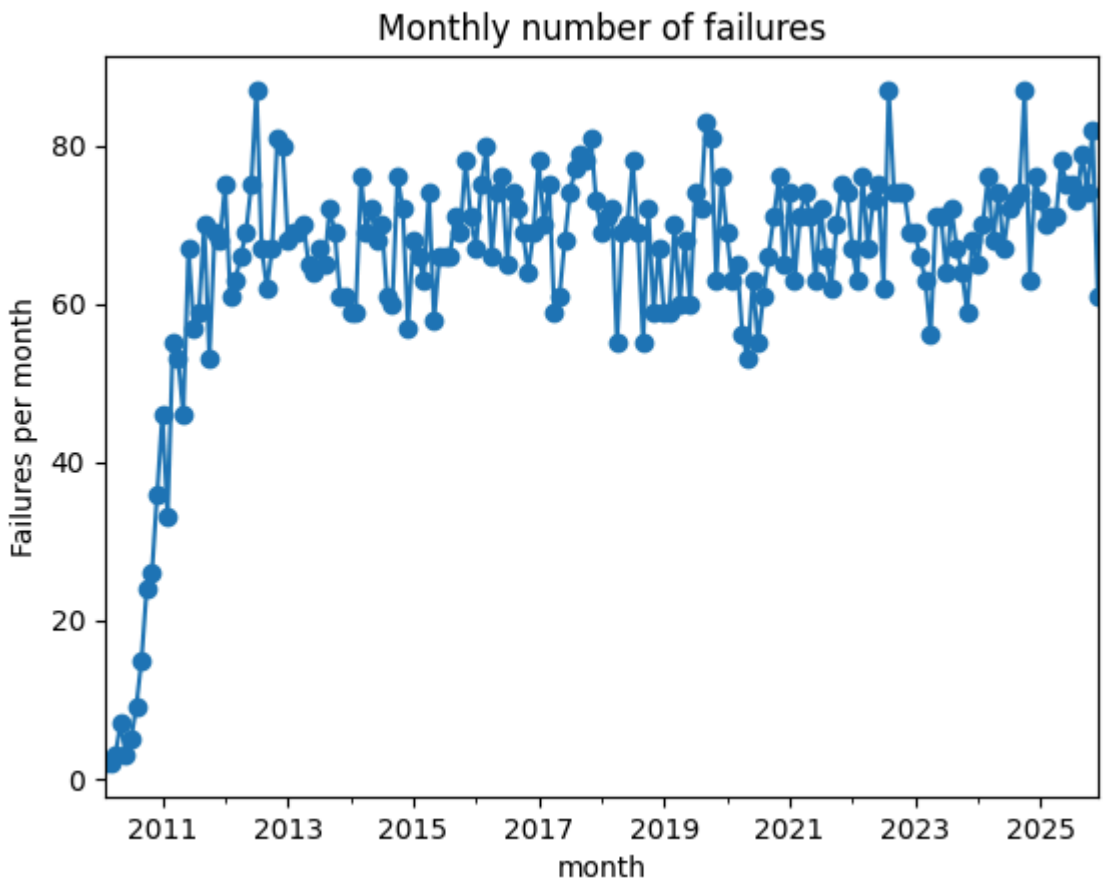
III. Experiments

A. Exploratory Analysis

ystem_id	component_id	event_date	event_time	event_type	event_report	system_age	component_age	usage_since_last_event_h	FF	cost_event	cost_cumulated
1	1	2010-01-08	04:43:11	inspection	NaN	172.72	172.72	172.72	NaN	100	100
1	1	2010-01-15	23:01:11	inspection	NaN	359.02	359.02	186.30	NaN	100	200
1	1	2010-01-23	04:42:53	inspection	NaN	532.71	532.71	173.69	NaN	100	300

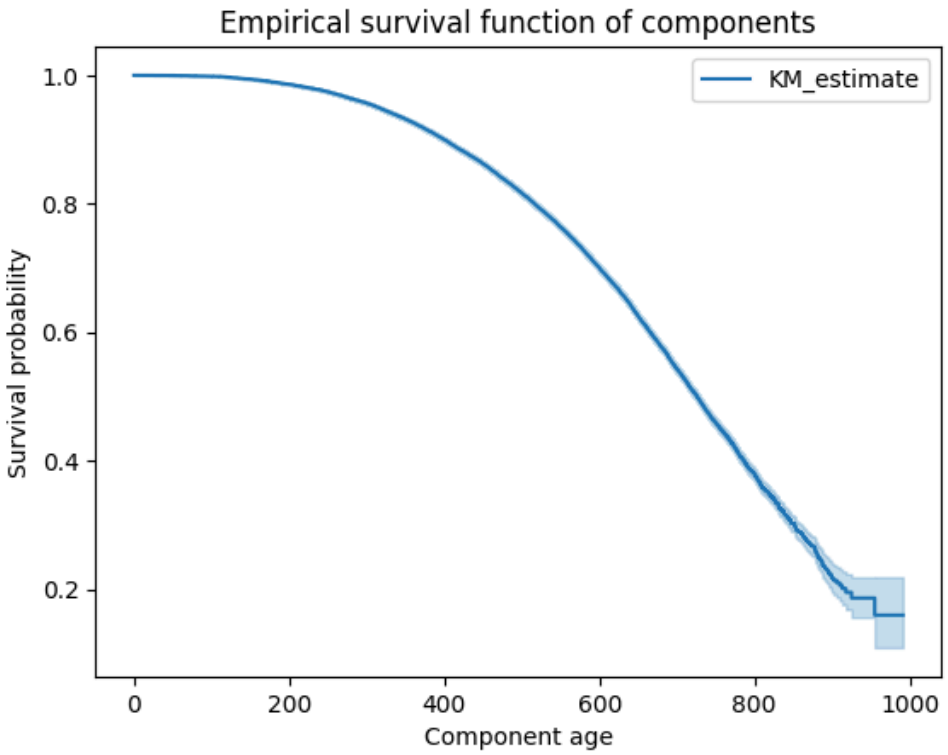


This graph shows the evolution of the number of events per month over time. At the beginning of the period, the number of events increases rapidly, indicating a transient phase where the system progressively reaches its operational regime. After this initial growth, the number of events stabilizes around a relatively constant level between roughly 500 and 560 events per quarter, with some natural fluctuations. This stabilization suggests that the system reaches a steady state in which events occur at a fairly regular rate over time.

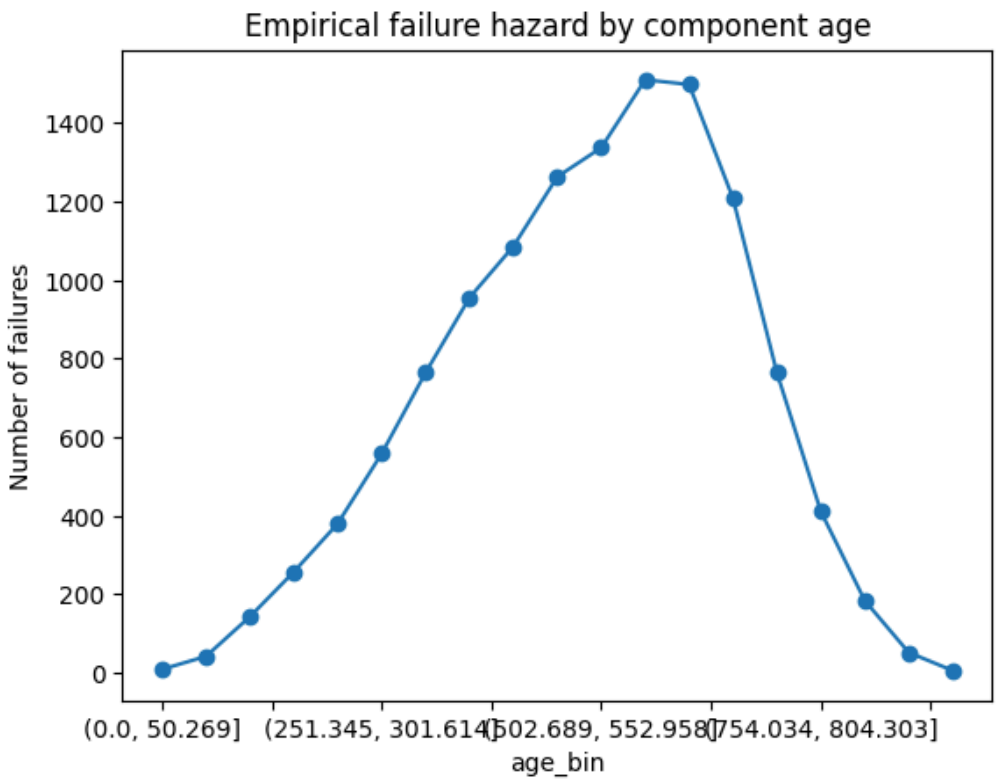


Here we take a look at the evolution of the monthly number of failures over time. Initially, the number of failures increases rapidly, reflecting a transition period as the system reaches its normal operating regime. After this phase, the number of failures stabilizes around a relatively constant level, generally between about 60 and 80 failures per month, with some natural fluctuations. This indicates that the system reaches a steady state where failures occur at a fairly regular rate over time.

This graph is consistent with the previous one showing the number of events per quarter. In both cases, we observe an initial growth phase where the system progressively reaches its operating regime, followed by a stabilization period. Once the system reaches this steady state, the number of failures fluctuates around a relatively constant level, which explains the stable number of events observed over time.



- The graph shows the survival probability of a component decreases steadily with age.
- Most components have a high probability of survival at a young age.
- Beyond 600, the probability of survival declines rapidly



- This graph shows the number of failures as a function of component age.

- Failures are rare at a young age, then increase strongly with age.
- After the peak, the number of failures decreases. (Likely because very few components survive)

These two last graphs describe the same aging process from different perspectives. The survival curve shows that the probability of a component remaining operational decreases as it ages, with a sharp decline after around 600 hours. Consistently, the second graph shows that failures are rare at a young age but become more frequent as components get older. After reaching a peak, the number of failures decreases because most components have already failed, leaving fewer surviving components that can fail later.

In summary, component aging leads to a decreasing survival probability and an increasing failure rate, highlighting the strong relationship between age and reliability.

A more detailed and complete analysis is available in the github via this link : https://github.com/alc4ml/ProjetM2D2526/blob/simulator/Data_Analysis.ipynb

B. Parameter Impacts

Dans cette partie, nous étudions l'influence des différents paramètres du modèle sur les indicateurs de performance d'un système de maintenance simulé. L'objectif est de comprendre comment certains paramètres du modèle influencent le comportement global du système, en particulier en termes de coût, de fréquence des pannes et de qualité de détection.

Le projet global vise à simuler la dégradation d'une flotte de systèmes contenant des composants susceptibles de tomber en panne. Le simulateur permet de générer des données simulées représentant les inspections, les réparations préventives, les remplacements et les pannes des composants.

Dans ce contexte, l'étude de sensibilité des paramètres constitue une étape importante du projet. Elle permet d'identifier les paramètres les plus influents et d'apporter des éléments d'aide au réglage d'une politique de maintenance.

Plus précisément, nous analysons l'effet de plusieurs paramètres sur différents indicateurs de performance :

- le coût moyen de maintenance ;
- le nombre moyen de pannes ;
- la qualité de détection des inspections (précision et rappel) ;
- l'efficacité de la maintenance préventive

Configuration de la simulation

Les simulations sont réalisées sur un horizon d'un an. Le cadre général retenu pour l'étude est le suivant :

- horizon de simulation : **8760 heures (1 an)** ;
- nombre de systèmes simulés : **200** ;

- nombre de répétitions Monte Carlo : **25**.

Les coûts utilisés dans le modèle sont les suivants :

- coût de remplacement d'un composant : **100**
- coût d'une panne : **1200**

Les indicateurs de performance utilisés dans l'étude sont notamment :

- le coût moyen par système ;
- le nombre moyen de pannes ;
- le nombre moyen de remplacements ;
- la précision ;
- le rappel ;
- le ratio d'efficacité préventive (PER) ;
- un indicateur de stabilité.

Ces indicateurs sont calculés à partir des données simulées produites par le simulateur.

Méthodologie de simulation

Afin d'évaluer les performances du système pour différents paramètres, une approche de simulation Monte Carlo est utilisée.

Une simulation élémentaire est réalisée par une fonction exécutant le simulateur pour un ensemble donné de paramètres. Cette simulation génère les événements suivants :

- inspections,
- réparations préventives,
- remplacements,
- pannes.

Les indicateurs de performance sont ensuite calculés à partir de ces données simulées.

Afin d'obtenir des estimations robustes, plusieurs simulations indépendantes sont exécutées pour chaque configuration de paramètres. Les résultats sont ensuite agrégés afin de calculer les valeurs moyennes des indicateurs.

Deux types d'analyses sont ensuite réalisés :

- **une analyse de sensibilité unidimensionnelle**, dans laquelle un seul paramètre varie tandis que les autres restent constants ;
- **une analyse de sensibilité bidimensionnelle**, dans laquelle deux paramètres varient simultanément.

Ces analyses permettent d'identifier les paramètres les plus influents et d'observer les interactions possibles entre eux.

Analyse de sensibilité des paramètres

Nous analysons l'effet de plusieurs paramètres du modèle :

- le **seuil de détection** ;
- l'**intervalle moyen entre inspections** ;
- la **variabilité des inspections** ;
- la **variabilité du processus de dégradation**.

L'objectif est de comprendre comment ces paramètres influencent les performances du système de maintenance.

Scénario de référence

Avant d'étudier l'effet des paramètres, un **scénario de référence** est défini afin de servir de point de comparaison.

Les valeurs moyennes obtenues pour ce scénario sont les suivantes :

- coût moyen : **≈ 25 800**
- pannes moyennes : **≈ 1.7 par mois**
- remplacements moyens : **≈ 2.1 par mois**
- précision : **0.845**
- PER : **1.157**

Ces résultats montrent que la configuration de base du système fournit des performances raisonnables. Cependant, certaines variabilités importantes subsistent, notamment sur le coût. Le scénario de référence doit donc être considéré comme une base de comparaison plutôt que comme un réglage optimal.

Effet du seuil de détection

Le **seuil de détection** représente la sensibilité du système de diagnostic utilisé lors des inspections.

Lorsque ce seuil augmente, plusieurs tendances apparaissent :

- le ratio d'efficacité préventive (PER) diminue ;
- la précision du détecteur chute fortement ;
- le nombre de pannes tend à augmenter ;
- le rappel diminue.

Ces résultats s'expliquent par le fait qu'un seuil élevé retarde le déclenchement des actions de maintenance préventive. Les composants sont alors laissés plus longtemps en état de dégradation avant intervention, ce qui augmente le risque de panne.

Les résultats suggèrent que les zones les plus favorables se situent pour des valeurs de seuil faibles à intermédiaires.

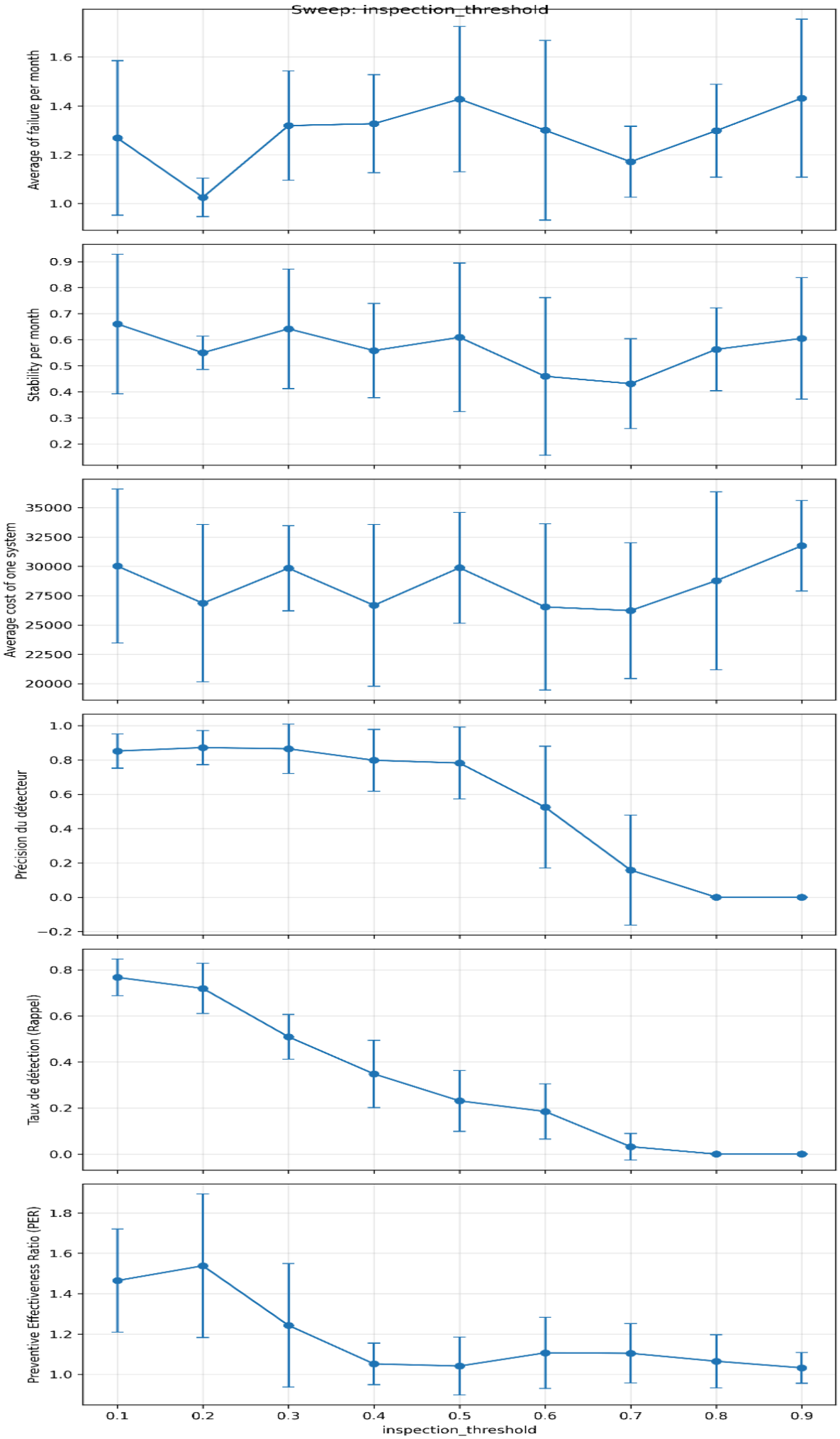

Sweep: inspection_threshold
Average of failure per month
Stability per month
Average cost of one system
Précision du détecteur
Taux de détection (Rappel)
Preventive Effectiveness Ratio (PER)
inspection_threshold

Figure: Effet du seuil de détection sur les indicateurs

Effet de l'intervalle d'inspection

Le paramètre μ représente l'intervalle moyen entre deux inspections successives.

Lorsque cet intervalle augmente, plusieurs tendances apparaissent :

- les pannes restent relativement faibles pour des valeurs intermédiaires ;
- le coût moyen diminue lorsque les inspections deviennent plus espacées ;
- la précision augmente avec l'intervalle d'inspection ;
- le rappel diminue.

Ces résultats mettent en évidence un compromis classique entre coût et qualité de détection. Des inspections trop fréquentes augmentent le coût global, tandis que des inspections trop espacées peuvent réduire la capacité du système à détecter certaines situations critiques.

Les résultats suggèrent l'existence d'une zone de compromis plutôt qu'un optimum unique.

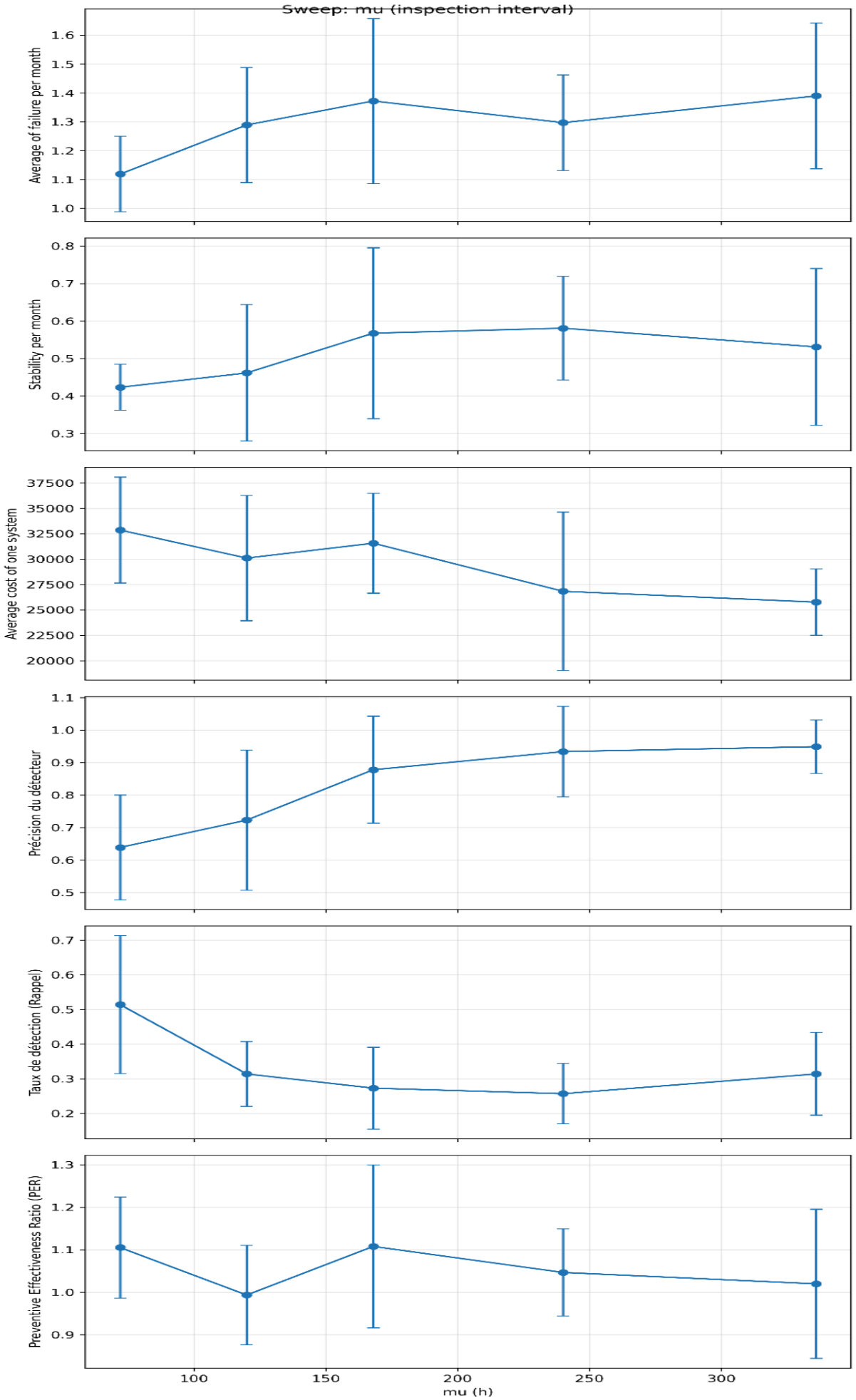

Sweep: mu (inspection interval)
Average of failure per month
Stability per month
Average cost of one system
Précision du détecteur
Taux de détection (Rappel)
Preventive Effectiveness Ratio (PER)
mu (h)

Figure: Influence de l’intervalle d’inspection

Effet de la variabilité des inspections

La variabilité des inspections représente les fluctuations autour de l’intervalle moyen entre inspections.

Lorsque cette variabilité augmente :

- la précision diminue fortement ;
- le rappel varie peu ;
- le coût et les pannes évoluent sans tendance claire.

Une augmentation de cette variabilité signifie que les inspections deviennent moins régulières et donc plus incertaines. Cela perturbe la qualité du processus de détection.

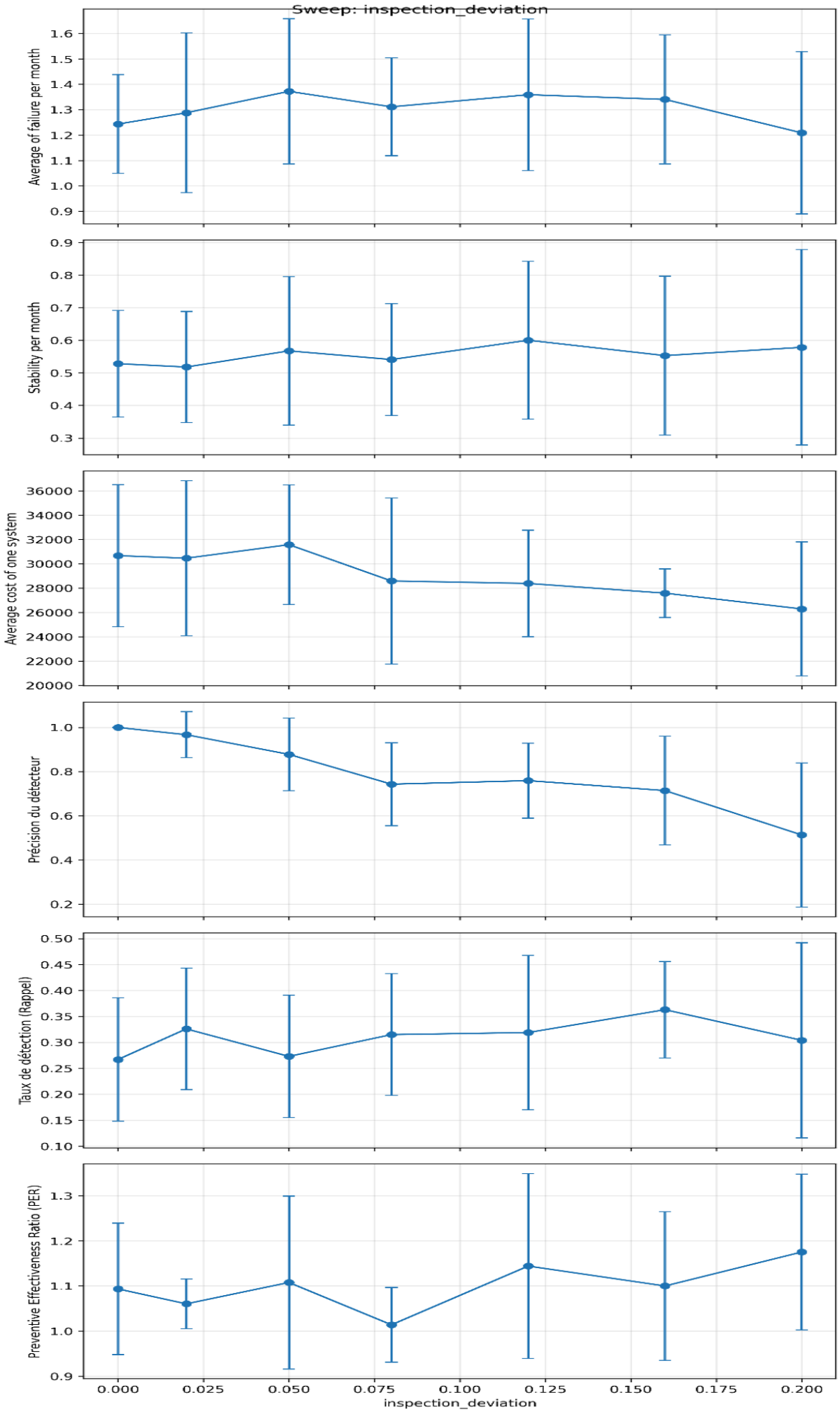

Sweep: inspection_deviation
Average of failure per month
Stability per month
Average cost of one system
Précision du détecteur
Taux de détection (Rappel)
Preventive Effectiveness Ratio (PER)
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
0.200
inspection_deviation

Figure: Effet de la variabilité des inspections

Effet de la variabilité de la dégradation

Le paramètre σ contrôle la dispersion temporelle du processus de dégradation.

Lorsque cette variabilité augmente :

- la précision diminue globalement ;
- les pannes ont tendance à augmenter ;
- le coût fluctue.

Ces résultats indiquent qu'une forte variabilité dans le processus de dégradation rend la détection moins fiable.

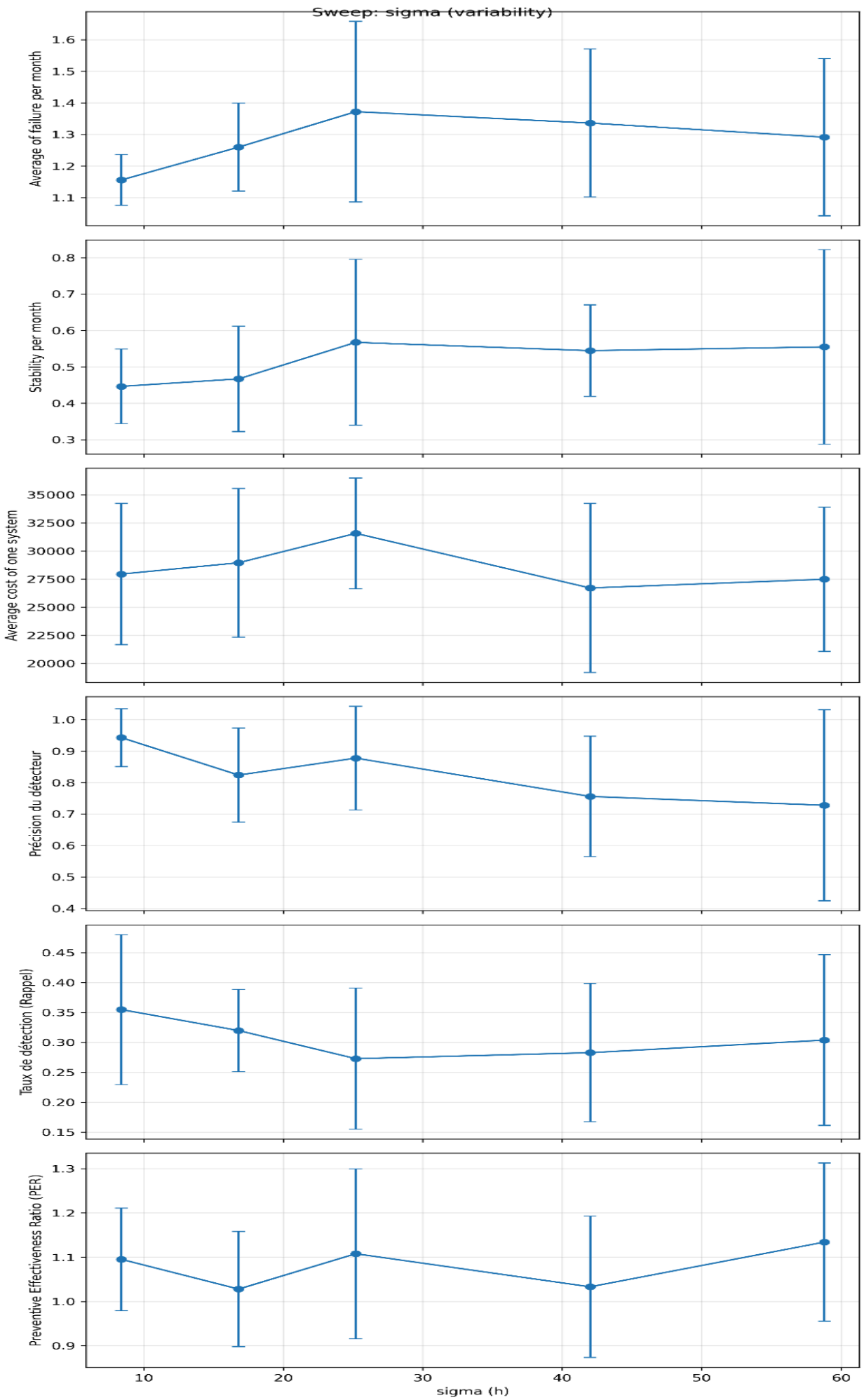

Sweep: sigma (variability)
Average of failure per month
Stability per month
Average cost of one system
Précision du détecteur
Taux de détection (Rappel)
Preventive Effectiveness Ratio (PER)
sigma (h)

Figure: Influence de la variabilité de la dégradation

Au-delà de l'influence individuelle des paramètres, il est également important d'étudier leurs interactions.

Les analyses bidimensionnelles permettent d'identifier les zones de l'espace des paramètres dans lesquelles les performances du système sont les plus favorables.

Intervalle d'inspection et seuil de détection : effet sur le coût

L'analyse conjointe de l'intervalle d'inspection et du seuil de détection montre que le coût dépend fortement de la combinaison de ces deux paramètres.

Les zones les moins coûteuses apparaissent pour des valeurs intermédiaires du seuil et pour des intervalles d'inspection relativement élevés.

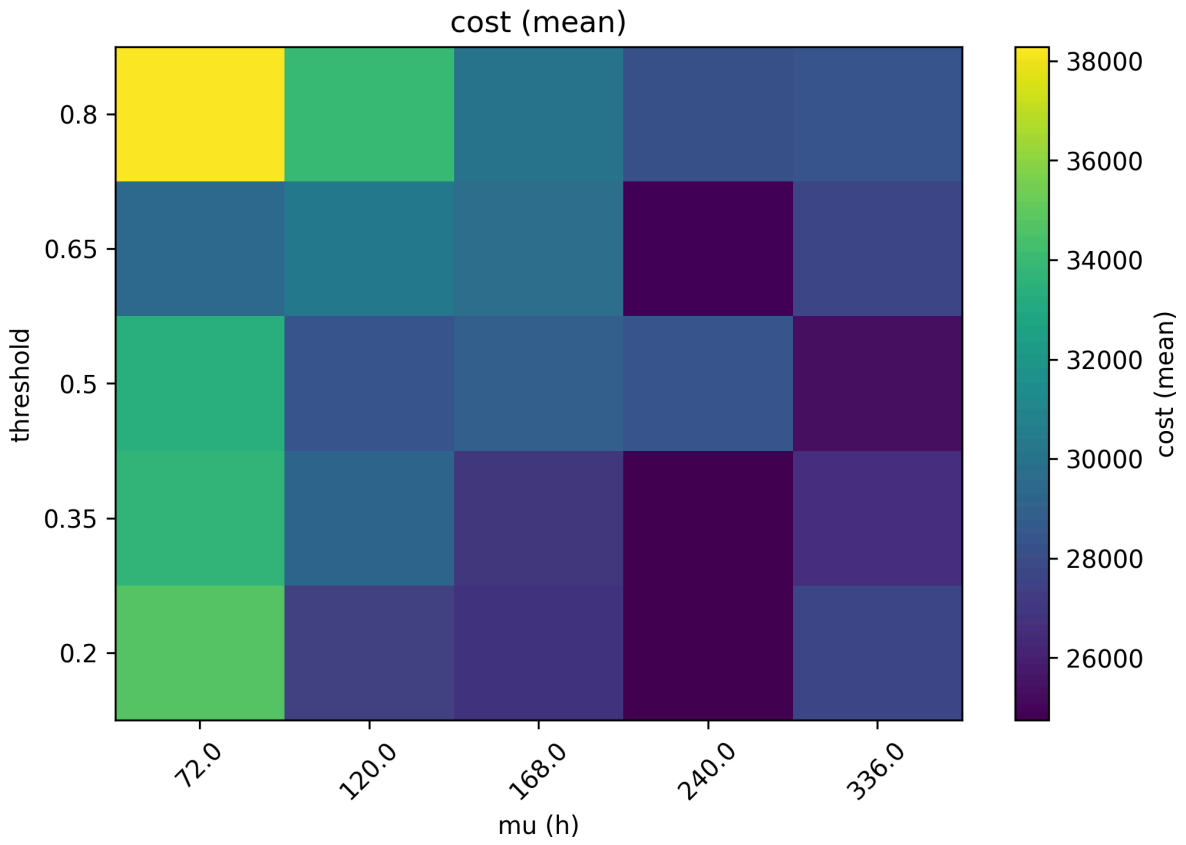


Figure: Coût moyen selon l'intervalle d'inspection et le seuil de détection

Intervalle d'inspection et seuil de détection : effet sur les pannes

La carte de chaleur des pannes montre une structure complémentaire à celle observée pour le coût.

Les pannes deviennent plus fréquentes lorsque les inspections sont espacées et que le seuil de détection est élevé.

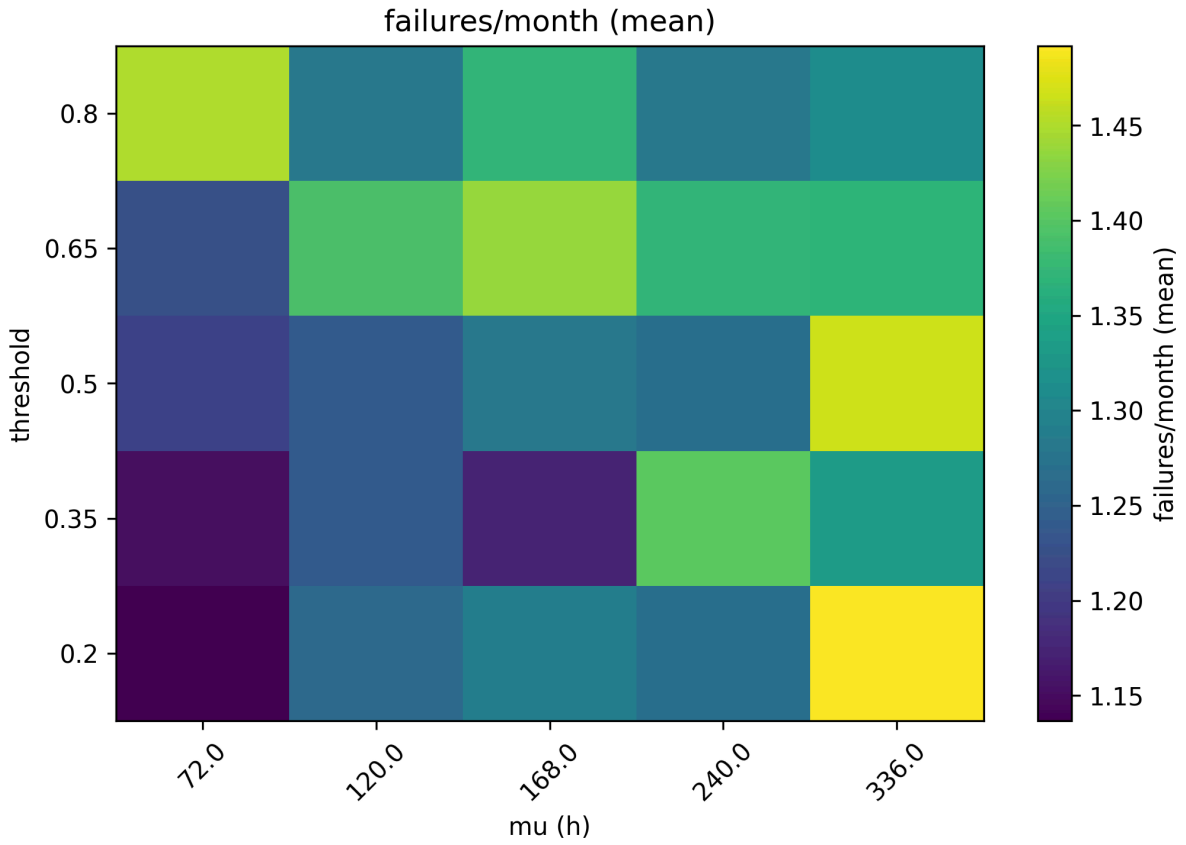


Figure: Nombre moyen de pannes selon l'intervalle d'inspection et le seuil

Variabilité des inspections et seuil de détection : effet sur le rappel

L'analyse conjointe de la variabilité des inspections et du seuil de détection montre que le rappel dépend principalement du seuil.

Lorsque le seuil augmente, le rappel chute fortement, tandis que l'effet de la variabilité des inspections reste plus modéré.

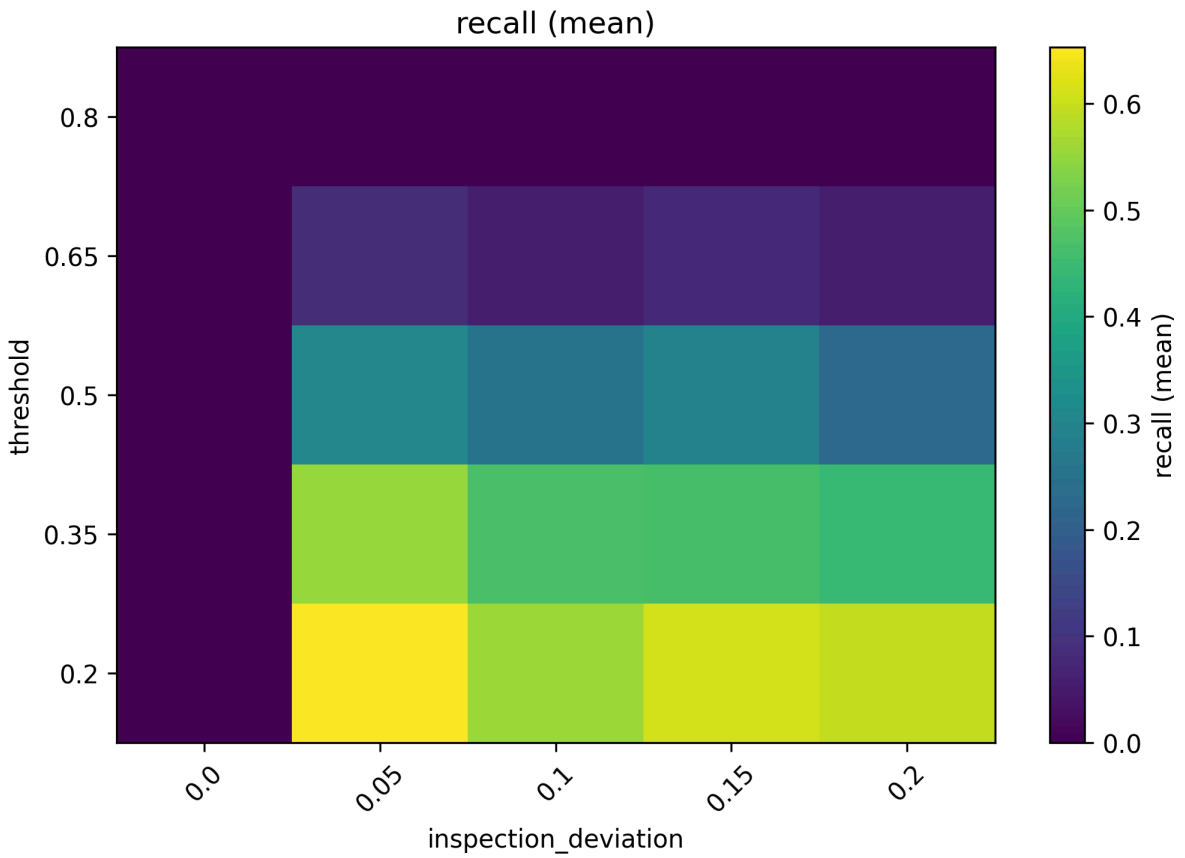


Figure: Rappel selon la variabilité des inspections et le seuil

Variabilité des inspections et seuil de détection : effet sur le coût

L'analyse du coût montre une structure plus complexe. Plusieurs mécanismes interviennent simultanément :

- coût des inspections ;
- coût des remplacements préventifs ;
- coût des pannes ;
- coût des actions correctives.

Certaines configurations avec une variabilité modérée peuvent conduire à des coûts plus faibles.

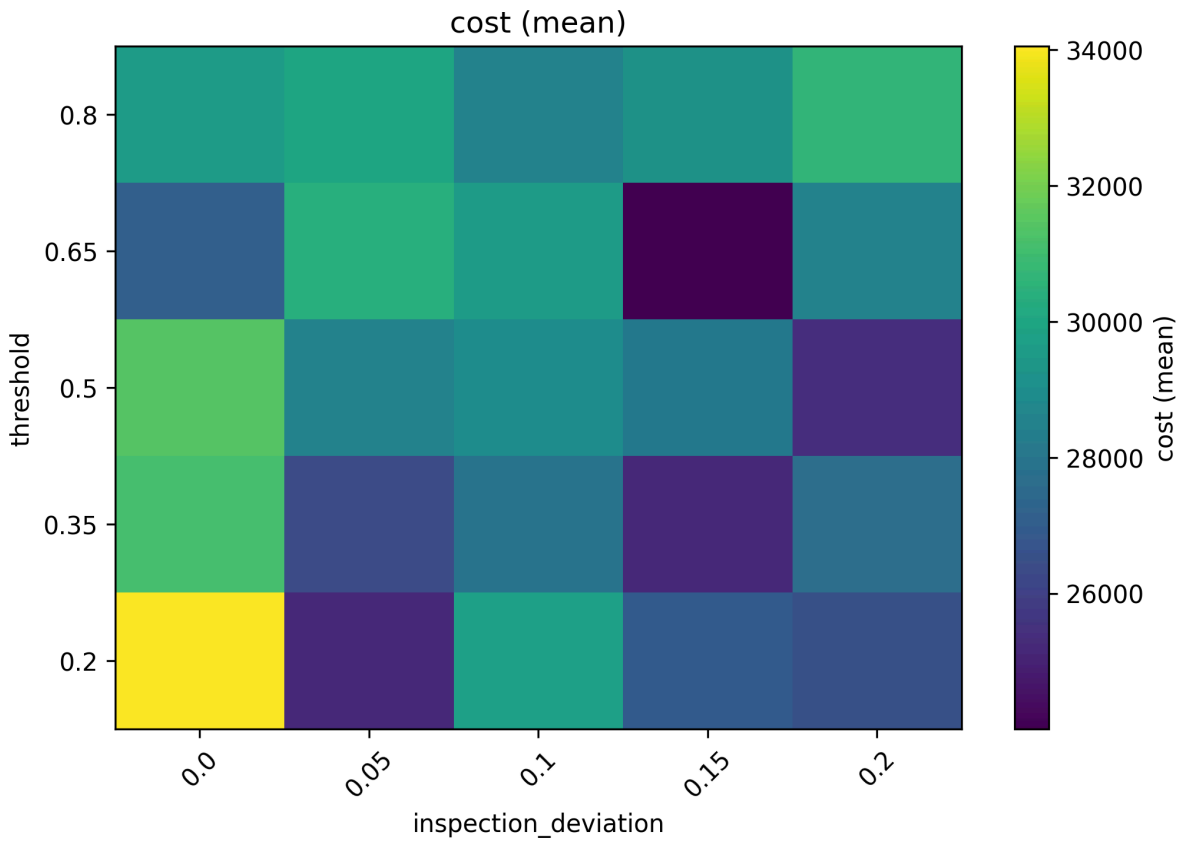


Figure: Coût moyen selon la variabilité des inspections et le seuil

Les résultats montrent que plusieurs paramètres influencent fortement les performances du système de maintenance.

Le **seuil de détection** apparaît comme un paramètre particulièrement important. Un seuil trop élevé détériore les indicateurs de détection et augmente la fréquence des pannes.

Le paramètre correspondant à la **fréquence des inspections** introduit quant à lui un compromis entre coût et qualité de détection.

Les paramètres de **variabilité temporelle** affectent surtout la robustesse du processus de surveillance.

Enfin, les analyses bidimensionnelles mettent en évidence des interactions importantes entre les paramètres, ce qui souligne l'intérêt d'une approche conjointe pour le réglage d'une politique de maintenance.

Cette étude a permis d'analyser l'effet de plusieurs paramètres du modèle sur les indicateurs de performance d'un système de maintenance simulé.

Les résultats montrent que :

- le seuil de détection joue un rôle central dans les performances du système ;
- la fréquence des inspections introduit un compromis entre coût et qualité de détection ;

- la variabilité du processus influence la robustesse du système ;
- les interactions entre paramètres sont importantes.

Ces résultats contribuent à une meilleure compréhension du comportement du système et constituent une étape vers l'optimisation des politiques de maintenance.

IV. Model Inversion

Retrouver les paramètres d'usure et connaître les données, la politique de maintenance et l'efficacité du détecteur.

V. Mathematical Description of Models - Oumar

VI. Parametric Estimation

A. Statistical Validation and Parameter Estimation

To prove that the simulator works correctly mathematically, we performed parameter estimation on the generated output logs (sample_system_dataset_0.csv) and (sample_system_dataset_1.csv). The goal is to reverse-engineer the logs to find the original input parameters using **Maximum Likelihood Estimation (MLE)**.

1/ Inspection Intervals (Normal Distribution)

The time between inspections is modeled using a Normal Distribution. By filtering the logs for consecutive inspections and calculating the mean and standard deviation of the time deltas, we successfully recovered:

- **Calculated μ:** 166.54 hours and 166.13 hours (True input: 168)
- **Calculated σ:** 26.70 hours and 27.24 hours (True input: 25.2)

2/ Failure Ages (Weibull Distribution and Right-Censoring)

The component's natural failure age follows a Weibull distribution characterized by a shape parameter β and a scale parameter η.

The Mathematical Trap (Survival Bias):

If we only calculate the parameters based on the components that actually failed, the math yields incorrect results (e.g., $\eta = 575$ instead of 720). This happens because of **Right-Censoring**: the inspections successfully remove worn-out components *before* they fail. By ignoring these "surviving" components, the naive model assumes components die much younger than they actually would.

The Solution:

To correctly estimate the parameters, the Likelihood function L must account for both failures (using the Probability Density Function, $f(t)$) and preventive replacements (using the Survival Function, $S(t)$):

$$L(\eta, \beta) = \prod_{\text{failures}} f(t_i; \eta, \beta) \times \prod_{\text{replaced}} S(t_j; \eta, \beta)$$

Using a custom Python script optimizing this negative log-likelihood, we successfully recovered the exact parameters:

- **Calculated η:** 720.16 hours and 721.06 (True input: 720)
- **Calculated β:** 3.00 and 2.96 (True input: 3.0)

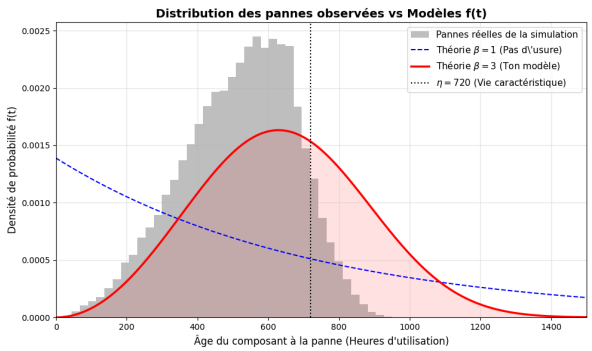
B. Visualizing Theoretical vs. Observed Data

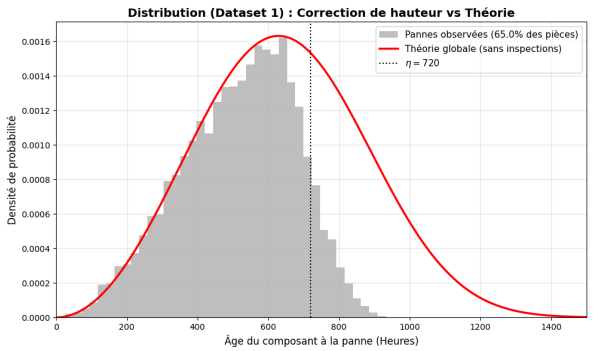
To visually validate the simulator, we plotted the histogram of the actual failure ages generated by the simulation against the theoretical Weibull Probability Density Function $f(t)$.

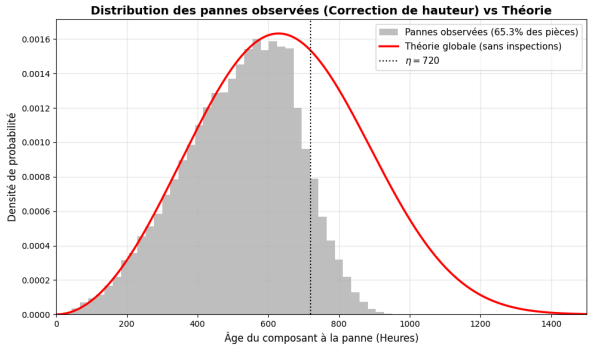
$$f(t) = \frac{\beta}{\eta} \left(\frac{t}{\eta} \right)^{\beta - 1} e^{-(t/\eta)^{\beta}}$$

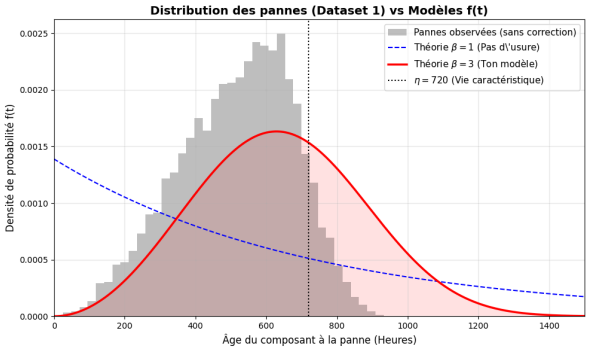
Height Correction (Accounting for Preventive Maintenance):

Because a large portion of components are replaced preventively (e.g., only 30% to 66% actually fail depending on the dataset), the histogram's area must not be forced to 100%. We adjusted the weights of the histogram bins so that the total area equals the actual *failure rate* of the simulation. This allows the observed data to fit perfectly under the theoretical curve.









VII. Policy Optimization

Objective: Automatically find the optimal values of the controllable parameters that minimize the **`Cost`**.

Let θ = (μ, σ, θ, inspection_deviation, inspection_threshold) ∈ $\mathbb{R}^5$ be the vector of controllable parameters.

A. Cost Function C(θ)

• C(θ) is the cost computed from a unique simulation $C(\theta) = w_1 \times c_1 + w_2 \times c_2$

• As the simulation is random, the true objective is $\bar{C}(\theta) = \mathbb{E}[C(\theta)]$

Is the objective differentiable ?

Discrete decision events

• Replacement decisions are binary:
`age_beta > threshold` or **`cdf > threshold`**

• Expiration check:
`age >= expiration`

Stochastic jumps

• Random draws cause discrete changes in event sequences (failure vs. no failure, replacement vs. no replacement).

Non-smooth transformations

• The cost is a sum over discrete events (inspections, replacements, failures), each triggered by inequalities.

Thus, the overall mapping θ → C(θ) is piecewise constant or has kinks. Because of these discontinuities and non-smoothness, no analytical gradient exists.

B. Monte-Carlo estimation of the cost

Since the simulator contains random draws, the observed cost varies across runs even for the same parameters. We approximate the expected cost with Monte-Carlo:

$\hat{C}(\theta) = (1 / M) \sum_{i=1}^{M} C_i(\theta)$

- Increasing M makes the estimate more stable (variance decreases roughly as 1/M),
- but increases computational time.

Risk-sensitive objective

In addition to the mean cost, we may penalize high variability (risk).
We use a mean-variance style objective:$J(\theta) = E[C(\theta)] + \alpha \cdot Std(C(\theta))$

With $\alpha \cdot Std(C(\theta))$ to penalize the variation.

- α is a tunable risk weight (**`alpha_std`** in the code).

C. Genetic Algorithm

Why a Genetic Algorithm (GA)?

As discussed, the mapping $\theta \rightarrow C(\theta)$ produced by the simulator is typically:

- non-linear
- non-convex
- not differentiable (discrete events, thresholds)
- noisy (stochastic simulation)

For these reasons, gradient-based methods are not reliable.

A Genetic Algorithm is a good choice because it is:

- derivative-free
- global (population-based exploration)
- robust to noise (especially with validation strategies like the one used here)

Genetic Algorithms (GA) are a family of optimization methods inspired by the mechanisms of natural selection. The most successful solutions are selected and combined to produce increasingly effective solutions over successive generations.

Another way to view them is as a structured search over a table of candidate solutions, where parameter values are recombined and modified in order to explore the search space and progressively discover better configurations.

Steps of a Genetic Algorithm

a) **Population Initialization**

We randomly generate a set of candidate solutions: { θ(1), θ(2), ..., θ(N) } where:

- **N** = population size (**`taille_pop`**)
- each parameter is randomly sampled within its allowed bounds

b) **Evaluation (Fitness)**

Each individual is evaluated using an objective function.

In our case: $J(\theta) = E[C(\theta)] + \alpha \cdot Std(C(\theta))$

c) **Selection**

We select the individuals that will reproduce. In our implementation, we use **tournament selection**. At each selection step:

- a subset of individuals is randomly sampled from the population
- the individual with the best fitness (lowest objective value) is selected

Formally, if **T** individuals are randomly drawn from the population { θ(1), θ(2), …, θ(N) }, the selected parent is : $\theta^* = \arg\min_{\theta \in T} J(\theta)$

This mechanism favors good solutions while still preserving some level of exploration in the search space.

d) **Crossover**

Two parents are combined to produce a new individual (child).

Example:

Parent A:
[400, 50, 30, 0.2, 0.5]

Parent B:
[300, 70, 15, 0.3, 0.7]

The child is created by combining the parameters of the two parents.
For example, one possible child could be:

Child:
[400, 70, 30, 0.3, 0.5]

This operation allows the algorithm to **recombine good characteristics from different solutions**, potentially producing better individuals in the next generation.

Formally, if two parents are θ_A and θ_B, the child θ_{child} is obtained by combining their parameters component-wise.

e) Mutation

A small random perturbation is introduced to explore new solutions.

Example: $\mu = 400 \rightarrow 410$

Mathematically, a parameter is modified by adding a random noise:

$\theta_j = \theta_j + N(0, \sigma)$ where:

- $N(0, \sigma)$ is a random value drawn from a normal distribution with mean 0 and standard deviation σ.

Mutation helps prevent the algorithm from getting stuck in a local minimum by introducing new variations in the population.

f) Elitism

The best individuals from the previous generation are kept unchanged. This guarantees that good solutions are not lost from one generation to the next.

In our code: elitism = 2

So, the 2 best individuals are preserved in the new generation.

Formally, if the population is ranked according to the objective function $J(\theta)$, the elite individuals are the ones with the lowest values of $J(\theta)$, and they are copied directly to the next generation.

g) Immigration

To maintain diversity in the population, some individuals are replaced by randomly generated solutions.

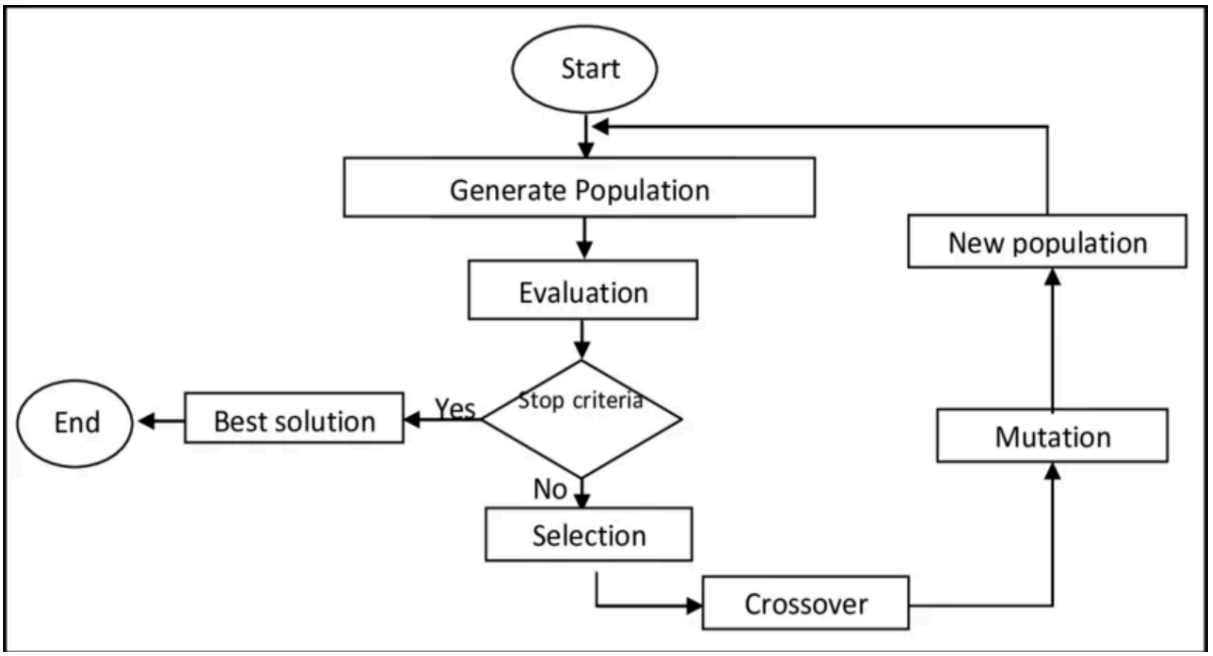
This mechanism introduces new genetic material into the population and helps the algorithm explore new regions of the parameter space.

h) New Generation

The new population is formed by combining: elites + children + immigrants

The process then repeats for the next generation: evaluation → selection → crossover → mutation

This iterative process continues until a stopping criterion is reached (for example a maximum number of generations or convergence of the objective function).



D. Two-Stage Evaluation (Fast and Validation Simulations)

Because the simulator is stochastic, the estimated cost depends on the number of Monte-Carlo simulations **M** used to approximate the expectation.

In the code, two different values of **M** are used:

- a **small M** for fast evaluation during the genetic algorithm
- a **large M** for accurate validation of promising solutions

Fast evaluation (small M)

During the optimization process, each candidate solution must be evaluated many times. Using a large number of simulations would make the algorithm extremely slow.

Therefore, we use a **small number of simulations** to obtain a quick approximation of the cost.

This makes the search much faster, although the estimate is noisier.

Validation evaluation (large M)

Once good candidate solutions are found, we re-evaluate them using a **larger number of simulations**.

This provides a much more accurate estimate of the expected cost.

Why this two-stage approach?

This strategy balances **speed and accuracy**:

- small M → fast exploration of the parameter space
- large M → reliable comparison between the best candidates

Without this approach, the algorithm would either:

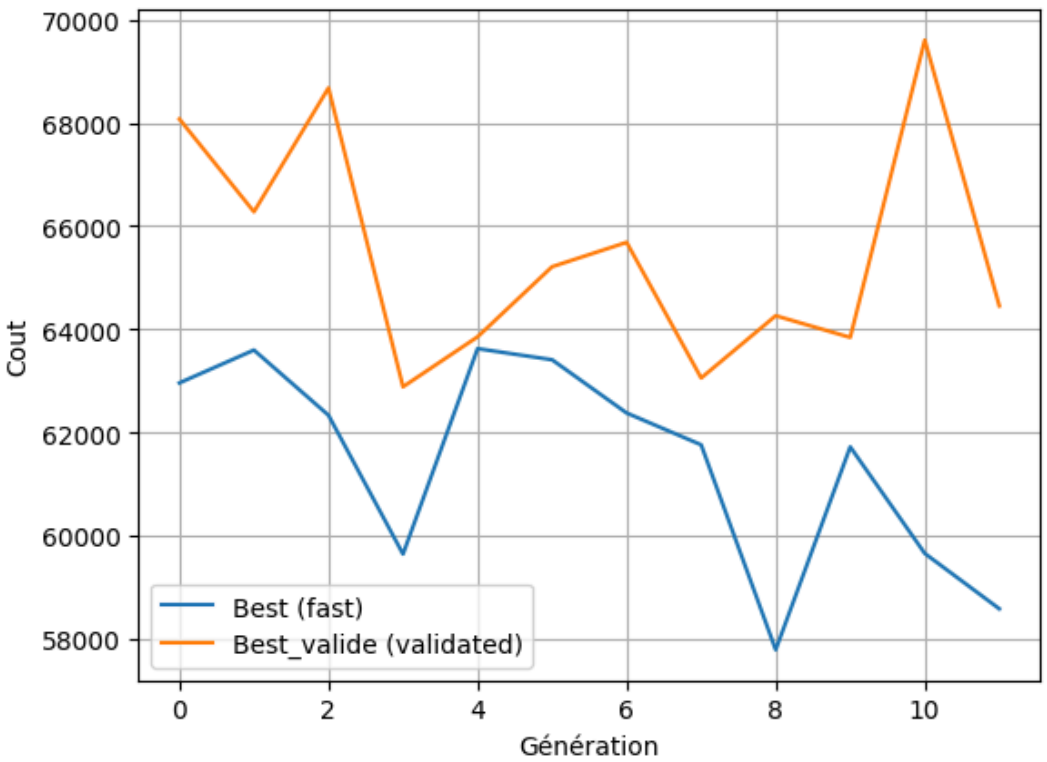
- be **too slow** (if we always used large M), or
- produce **unstable results** (if we always used small M).

By combining both, we obtain an efficient and robust optimization procedure.

Results

Initial Cost	**Optimized cost**
75059.30	65113.49

Gain = 13.25 %



This graph shows the evolution of the cost across generations of the genetic algorithm. The blue curve represents

the best individual of each generation evaluated with a small number of simulations, while the orange curve corresponds to the same solution re-evaluated with a larger number of simulations to obtain a more reliable estimate of the cost. The blue curve is more fluctuating, which is expected since the fast evaluation is noisier. The orange curve, although it still presents some variations due to the stochastic nature of the simulator, provides a more accurate representation of the true performance of the solutions. Overall, despite these oscillations, the validated cost tends to decrease over generations, indicating that the genetic algorithm progressively discovers better parameter configurations.

VIII. Pipeline and User Interface

A. Pipeline Library

The Pipeline of Marcel's Company M2 des Données (PipMCM2D). This library contains the necessary functionalities for a complete process of analysis and policy optimization for fleet maintenance. It makes use of the library SimMCM2D. Both libraries are available for installation using PIP through the project repository. The library includes functionalities for:

- **Parameter Estimation** from simulated datasets, which is done using the functions exposed by the library:
    - *estimate_inspection_parameters(df)*: returns estimations for mu and sigma
    - *estimate_component_parameters(df)*: returns estimations for eta and beta
    - *estimate_parameters(df)*: returns all parameters estimated
    - *estimate_parameters_filepath(filepath)*: returns all parameters estimated from a sample.csv filepath
- **Maintenance Policy Optimization** of using genetic algorithms, using the functions exposed by the library
    - *compute_cost: computes the average cost function using Monte-Carlo*
    - *algorithme_genetique*: optimizes the policy using Genetic Algorithms

As done in the previous library, use examples and further documentation are made available on the path *examples*:

- *parameter_estimator.ipynb*: simple examples on the use of the functionalities for parametric estimation
- *AG_Optimisation.ipynb*: an example of policy optimization using Genetic Algorithms
- *pipeline_example.ipynb*: a complete pipeline making use of a random sample file, followed by a parameter estimation step used for optimization

B. User Interface

Technologies

The user interface was developed using **Streamlit**, a Python framework for building interactive web applications for data science projects. The application is implemented in **Python** and uses libraries such as **pandas** for data manipulation and visualization libraries to display simulation results and descriptive analyses.

This technology allows the development of a lightweight and interactive interface where users can configure parameters, run simulations, and analyze the generated data.

Application Architecture

The application follows a modular architecture. The main entry point is `app.py`, which manages the navigation between the different pages. Each page is implemented in a separate module located in the `views` directory.

The interface is organized into four main pages: **Configuration**, **Simulation**, **Descriptive Analysis**, and **Estimation**, corresponding to the different steps of the workflow.

How to access the interface

There are two ways to use the app:

1. Locally

- Clone the repository: https://github.com/alc4ml/ProjetM2D2526/
- Go into the `streamlit_app` folder and run : **streamlit run Pipeline/UI/streamlit_app/app.py**

2. Online

- Access the app directly on streamlit app : https://projetm2d2526-m2d.streamlit.app/

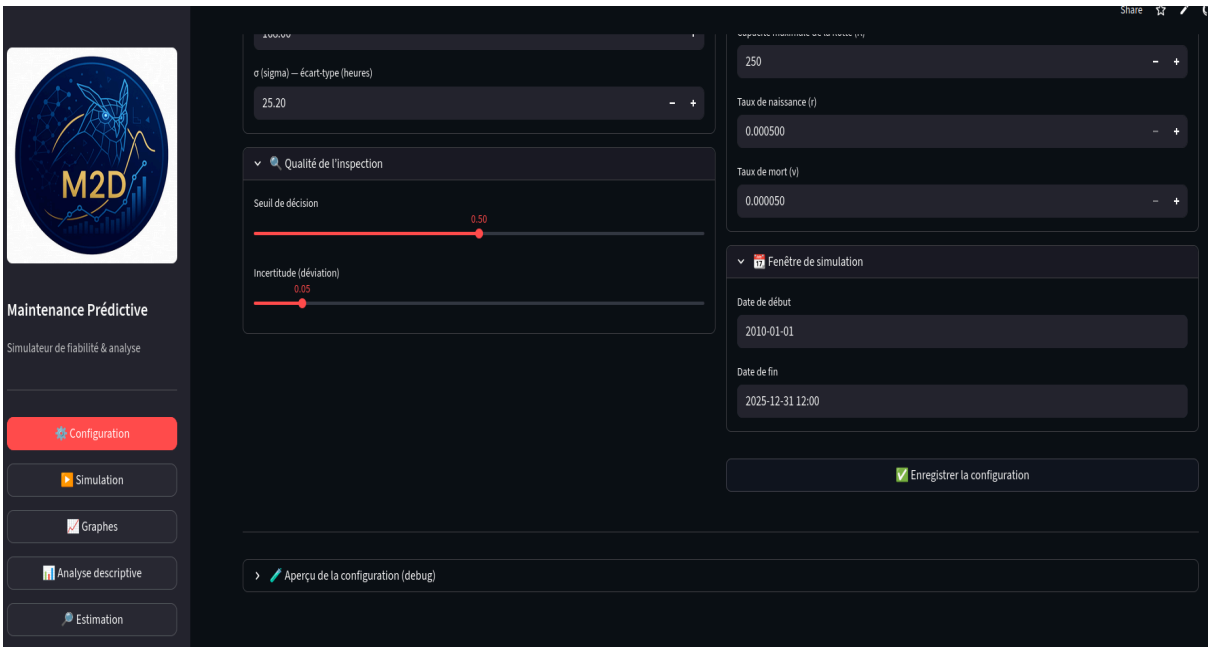
Navigation and User Worflow

Once you have accessed the app, you can :

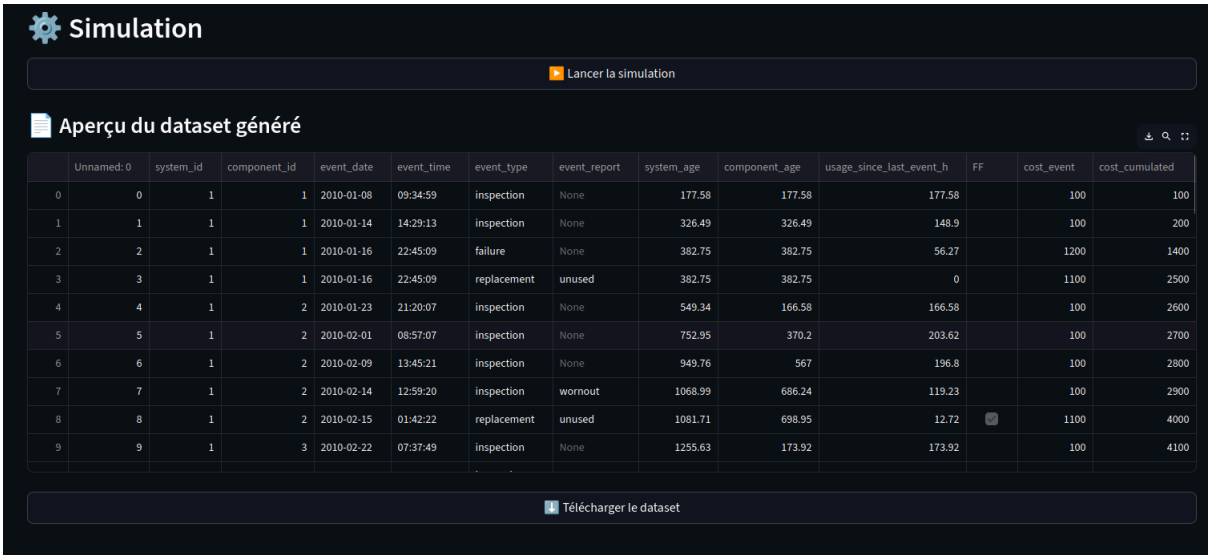
- Configure the parameters for simulations
- Run simulations
- Navigate through the app to view predefined KPI calculations
- Perform a descriptive analysis based on the generated dataset
- Look at the estimation of the parameters (find back their value using the dataset)

To do this, follow these steps :

- ❖ Go to the configuration page, choose the value of your parameters and then click on “Enregistrer la configuration” to save them :



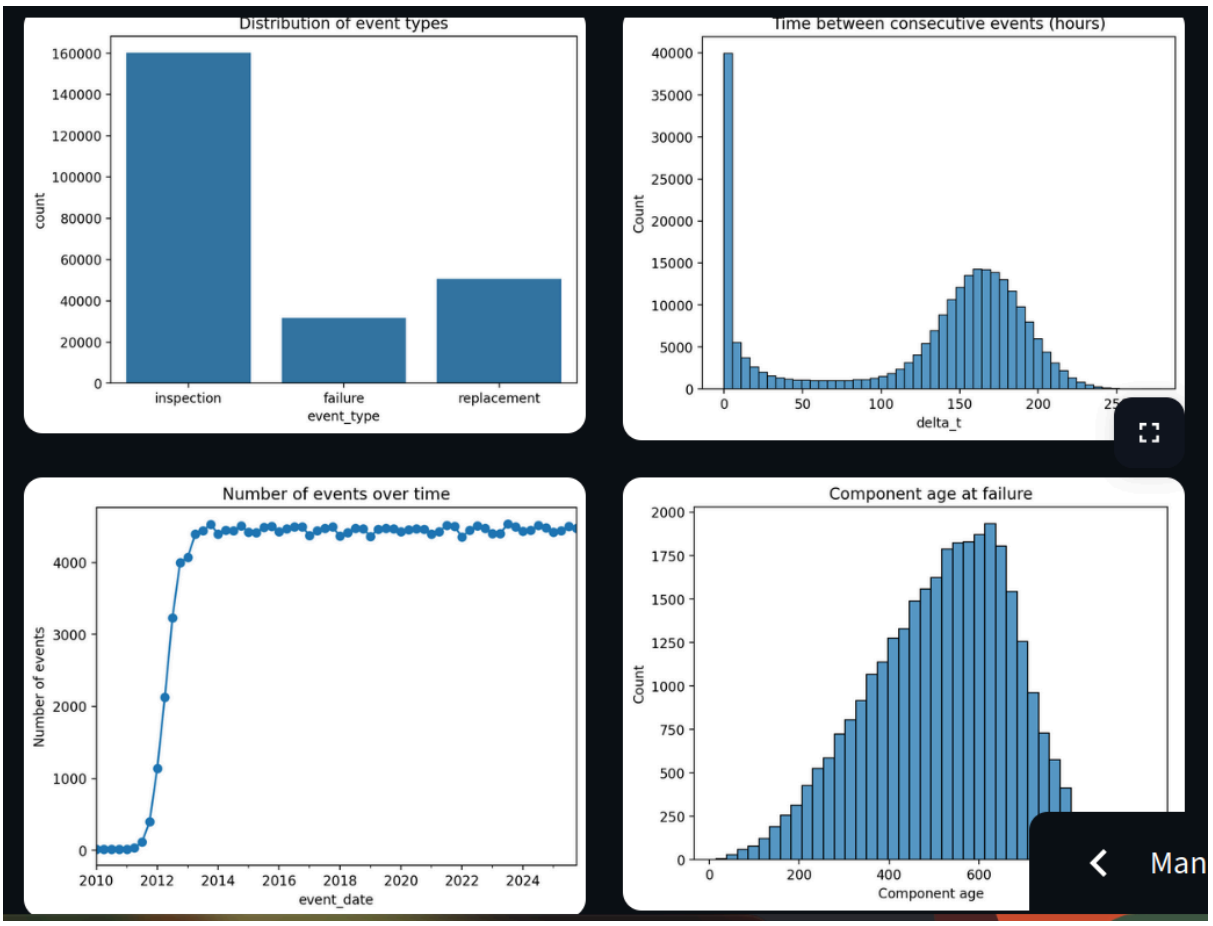
❖ Go to the simulation page and run a simulation :



	Unnamed: 0	system_id	component_id	event_date	event_time	event_type	event_report	system_age	component_age	usage_since_last_event_h	FF	cost_event	cost_cumulated
0	0	1	1	2010-01-08	09:34:59	inspection	None	177.58	177.58	177.58		100	100
1	1	1	1	2010-01-14	14:29:13	inspection	None	326.49	326.49	148.9		100	200
2	2	1	1	2010-01-16	22:45:09	failure	None	382.75	382.75	56.27		1200	1400
3	3	1	1	2010-01-16	22:45:09	replacement	unused	382.75	382.75	0		1100	2500
4	4	1	2	2010-01-23	21:20:07	inspection	None	549.34	166.58	166.58		100	2600
5	5	1	2	2010-02-01	08:57:07	inspection	None	752.95	370.2	203.62		100	2700
6	6	1	2	2010-02-09	13:45:21	inspection	None	949.76	567	196.8		100	2800
7	7	1	2	2010-02-14	12:59:20	inspection	wornout	1068.99	686.24	119.23		100	2900
8	8	1	2	2010-02-15	01:42:22	replacement	unused	1081.71	698.95	12.72	✓	1100	4000
9	9	1	3	2010-02-22	07:37:49	inspection	None	1255.63	173.92	173.92		100	4100

❖ Then, you can explore some predefined **descriptive analyses** available on the **“Analyse descriptive”** page.

An example is shown in the image below.



- Finally, the objective is to **estimate the parameters as if their true values were unknown**, using only the generated dataset. This can be seen on the **"Estimation"** page.

Deployment

The source code of the application is hosted on **GitHub**, which allows version control and collaborative development.

The application is deployed online using **Streamlit Cloud**, making it accessible directly through a web browser without requiring any local installation. This deployment solution simplifies the distribution of the application and allows users to easily interact with the interface.

IX. Appendix

C. Inspection Quality Law

The objective of an inspection is to check if a component is old, which can be seen as presenting wear signals. Since the age of a component is known, but not exactly its distribution, we model this phenomenon as follows:

A measure of the age/wear of a component will be an estimation of the Cumulated Density Function (CDF) of its failure age; if the CDF is close to 0, the component is new, and if it is close to 1, it almost certainly will present a pane.

Since the inspection is not perfect, we consider a distribution around the exact CDF in question. This is done by considering a Beta distribution whose average is the exact CDF:

$$Mean = CDF(component),$$

and variance given by

$$Var = \alpha * Mean * (1 - Mean),$$

where $0 < \alpha < 1$ is a quality measure, or Inspection Precision, this is due to a constraint of the Beta distribution that $Var < Mean * (1 - Mean)$.

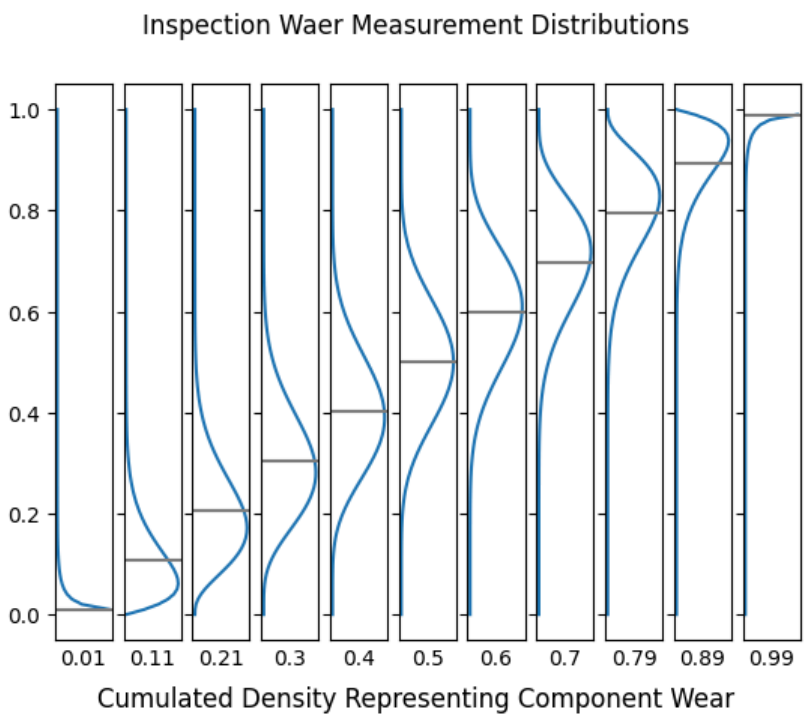
The parameters a, b of the Beta distribution are finally given as follows:

$$a = CDF * (CDF * (1 - CDF)/VAR - 1)$$

and

$$b = (1 - CDF) * (CDF * (1 - CDF)/VAR - 1).$$

This model results in densities around the exact CDF as follows:



For further discussion on the Beta Distribution and on the definition of the parameters from a mean and variance of choice, one may refer to https://en.wikipedia.org/wiki/Beta_distribution

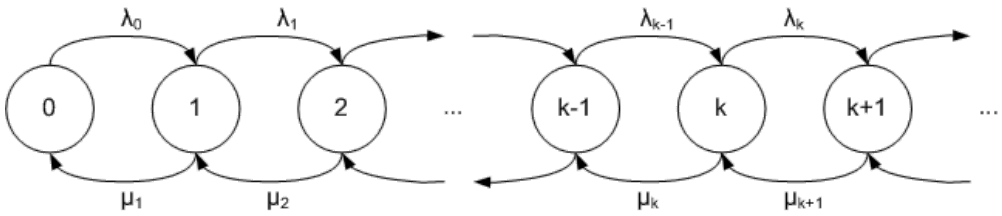
B. Fleet Evolution Law

We assume the fleets evolve following (approximately) a (stochastic) logistic model. See examples of real-world data suggesting such an assumption:

- Emirates: https://www.airfleets.net/chart/Emirates-fleet-chart.htm
- AirFrance: https://www.airfleets.net/chart/Air%20France-fleet-chart.htm

- GOL: https://www.airfleets.net/chart/GOL%20Transportes%20Aereos-fleet-chart.htm

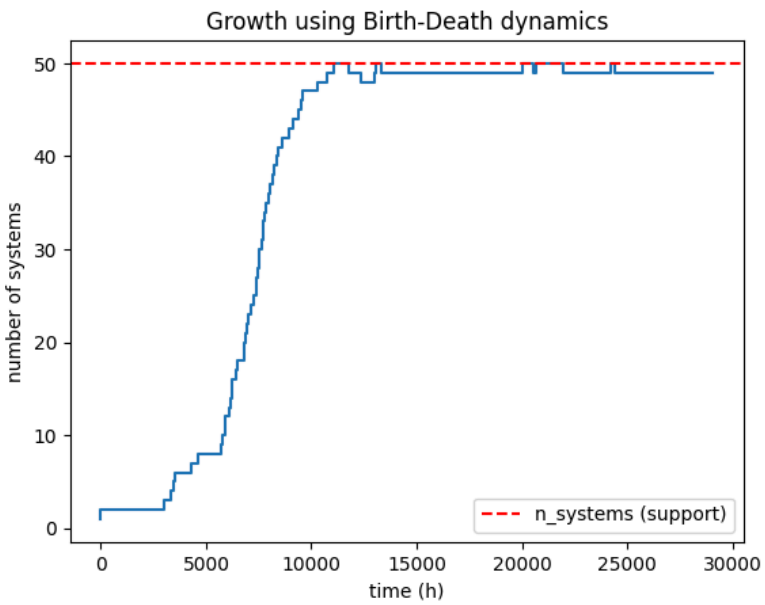
However, for discrete states (the natural number of systems), a more suitable approach is a Birth-Death Process, in continuous-time, that is, the time step is not fixed, but sampled at each time given a distribution; this follows a Continuous-Time Markov Chain model, see: https://en.wikipedia.org/wiki/Birth%E2%80%93death_process



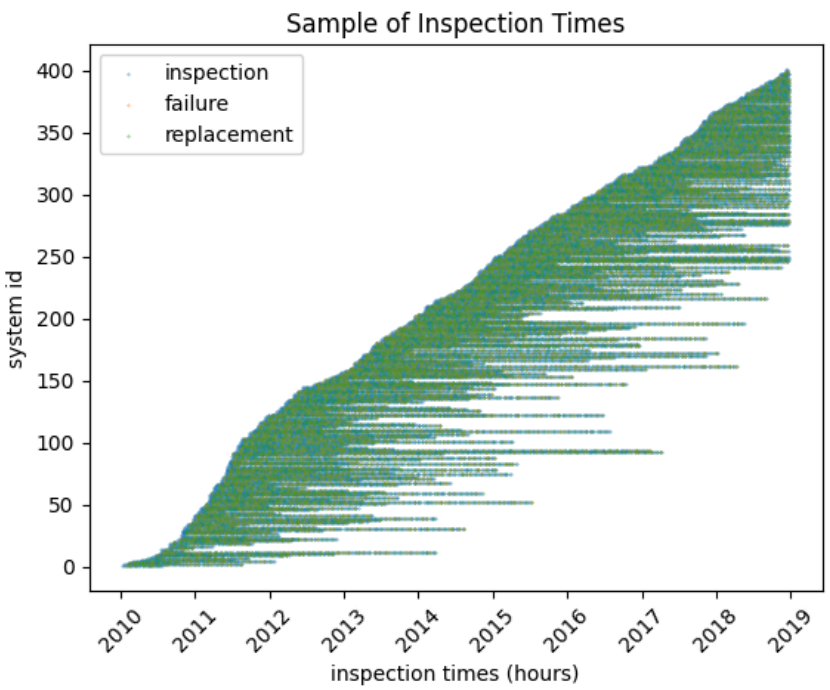
For our model, we consider the following birth rate to follow a logistic model, given the number n of systems:

$$\lambda_n = r\,n\,(1 - n/K)$$

where r is a growing factor, and K is the support capacity; the death rate $\nu_n = n * \nu$ is assumed to be constant per system, and describes the average rate at which a system is retired. We also impose that the population does not die, so a minimum of 1 system is maintained.



Further, on the following graph of events over time, one may observe the evolution of the fleet, and also that events for certain systems stop over time, signaling that the system has been removed:



For simulation, we implement a Gillespie algorithm. For further details on the algorithm, one may refer to: https://en.wikipedia.org/wiki/G illespie_algorithm.